Práctica guiada: del modelo legado al artefacto explícito

En la semana 3 usamos un payload **joblib** con tres claves: el estimador, los nombres de las features y la versión del modelo. En esta práctica vamos a separar responsabilidades:

- **model.joblib**: el objeto ejecutable;
- **manifest.json**: el contrato que permite saber si el objeto es compatible;
- código y pruebas: las reglas de validación y el comportamiento ante errores.

Resultado esperado

Al terminar tendrás un manifiesto mínimo escrito como JSON, una lista de invariantes y una matriz de fallos que otra persona podrá implementar en la segunda clase.

1. Inspeccionar el punto de partida

Primero comprueba qué columnas recibe el módulo de inferencia de la semana 3. No cambies todavía el modelo ni ejecutes predicciones.

```
from pathlib import Path
import csv

week_root = next(
    parent
    for parent in (Path.cwd(), *Path.cwd().parents)
    if (parent / "modules/04-model-packaging").is_dir()
)
input_path = week_root /
"assets/03-wine-quality/inference_samples.csv"
if not input_path.exists():
    input_path = week_root.parent /
"semana3/assets/03-wine-quality/inference_samples.csv"

with input_path.open(newline="", encoding="utf-8") as file:
    reader = csv.DictReader(file)
    rows = list(reader)
print(f"Fichero: {input_path}")
print(f"Columnas: {reader.fieldnames}")
print(f"Filas de ejemplo: {len(rows)}")
```

```
Fichero: C:\Users\pelay\Documents\COMILLAS\MLOPS\workspace\icai-
muiaap-operacion-de-modelos\semana3\assets\03-wine-quality\
inference_samples.csv
Columnas: ['sample_id', 'fixed_acidity', 'volatile_acidity',
'citric_acid', 'residual_sugar', 'chlorides', 'free_sulfur_dioxide',
```

```
'total_sulfur_dioxide', 'density', 'ph', 'sulphates', 'alcohol']
Filas de ejemplo: 5
```

2. Diseñar la frontera del artefacto

Completa esta tabla antes de escribir código. La pregunta clave es: **¿qué necesita un proceso de inferencia para decidir que puede utilizar el artefacto?**

Elemento	¿Dónde vive?	¿Qué riesgo evita?
Estimador entrenado	**model.joblib**	Tener que reentrenar o reconstruir el modelo en cada ejecución. Es el único elemento que requiere deserializar código.
Versión del esquema	**manifest.json** (`schema_version`); el valor esperado está en el código (`ARTIFACT_SCHEMA_VERSION`)	Interpretar un manifiesto con otra estructura como si fuera válido.
Orden y nombres de features	**manifest.json** (`feature_names`); la referencia es `FEATURE_NAMES` en `preprocess.py`	Construir un vector desordenado que el modelo acepta sin error y produce predicciones silenciosamente incorrectas.
Versión del preprocesado	**manifest.json** (`preprocessing_version`); se compara con `PREPROCESSING_VERSION`	Aplicar al modelo una transformación distinta de la usada en el entrenamiento.
Etiquetas de salida permitidas	**manifest.json** (`output_labels`); en el código, `OUTPUT_LABELS` y el tipo `QualityBand`	Exponer al consumidor una categoría que no espera (por ejemplo, `unknown`).
Versión del modelo	**manifest.json** (`model_version`)	Perder la trazabilidad, ya que cada predicción del CSV indica qué modelo la generó.
Tipo de estimador	**manifest.json** (`estimator_type`)	Cargar un objeto distinto del declarado. Permite saber qué contiene el bundle sin deserializarlo.
Reglas de validación	**Código y tests** (`ArtifactManifest`, `load_model_bundle`, `contracts.py`)	Que las reglas dependan de lo que diga el propio artefacto.

Justificación de la separación. `manifest.json` es texto plano y puede leerse y validarse sin ejecutar nada, mientras que `joblib.load` deserializa un objeto Python y puede ejecutar

código. Por eso el manifiesto se valida primero y el binario solo se carga si los metadatos son compatibles.

3. Práctica 2 — Construir un manifiesto mínimo

Completa el borrador de Python de la siguiente celda y ejecútalo para producir un JSON legible. Mantén los nombres de campo estables: el manifiesto es una interfaz entre quien empaqueta el modelo y quien lo ejecuta.

```
{
  "schema_version": "",
  "model_version": "",
  "preprocessing_version": "",
  "feature_names": [],
  "output_labels": [],
  "estimator_type": ""
}
```

Comprueba antes de continuar que:

- aparecen exactamente las seis claves obligatorias;
- `feature_names` tiene once nombres y no incluye `sample_id`;
- `output_labels` representa las categorías que la interfaz podrá devolver.

Decide también qué debe pasar si:

1. falta **manifest.json**;
2. aparecen features desconocidas o en un orden distinto;
3. el modelo devuelve una etiqueta que no está en **output_labels**;
4. una fila del CSV no cumple el contrato de entrada.

Respuestas

1. **Falta `manifest.json`**: se rechaza en la carga del bundle, antes de abrir `model.joblib` (`El manifiesto no existe: ...`).
2. **Features desconocidas o en otro orden**: se rechaza al validar el manifiesto, porque `feature_names` debe coincidir exactamente con `FEATURE_NAMES`. El estimador no llega a deserializarse.
3. **El modelo devuelve una etiqueta fuera de `output_labels`**: se rechaza en la inferencia, antes de construir la predicción (`quality_band no permitida por el manifiesto: 'unknown'`).
4. **Una fila no cumple el contrato de entrada**: se rechaza al validar la fila con `WineQualityRequest`; el error indica la fila, el `sample_id` y el campo, y no se escribe ningún CSV.

```
import json

feature_names = [
    "fixed_acidity",
```

```
    "volatile_acidity",
    "citric_acid",
    "residual_sugar",
    "chlorides",
    "free_sulfur_dioxide",
    "total_sulfur_dioxide",
    "density",
    "ph",
    "sulphates",
    "alcohol",
]

# No modifiques este orden: procede del contrato de la semana 3.
legacy_model_version = "wine-quality-rf-v1"  # model_version del
payload de S3

manifest_draft = {
    "schema_version": "wine-quality-bundle-v1",
    "model_version": legacy_model_version,
    "preprocessing_version": "wine-red-features-v1",
    "feature_names": feature_names,
    "output_labels": ["needs_review", "acceptable", "excellent"],
    "estimator_type": "RandomForestClassifier",
}

required_keys = {
    "schema_version",
    "model_version",
    "preprocessing_version",
    "feature_names",
    "output_labels",
    "estimator_type",
}

assert set(manifest_draft) == required_keys
assert len(manifest_draft["feature_names"]) == 11
assert "sample_id" not in manifest_draft["feature_names"]

print(json.dumps(manifest_draft, indent=2, ensure_ascii=False))
print("\nLa estructura es válida.")

{
  "schema_version": "wine-quality-bundle-v1",
  "model_version": "wine-quality-rf-v1",
  "preprocessing_version": "wine-red-features-v1",
  "feature_names": [
    "fixed_acidity",
    "volatile_acidity",
    "citric_acid",
    "residual_sugar",
```

```
    "chlorides",
    "free_sulfur_dioxide",
    "total_sulfur_dioxide",
    "density",
    "ph",
    "sulphates",
    "alcohol"
  ],
  "output_labels": [
    "needs_review",
    "acceptable",
    "excellent"
  ],
  "estimator_type": "RandomForestClassifier"
}

La estructura es válida.
```

4. Práctica 3 — Anticipar fallos

Antes de ver la demo del profesorado, completa la tabla. No describas solo el error: indica en qué frontera debería detenerse el flujo.

Cambio	Frontera que debe rechazarlo	Mensaje útil esperado	¿Se escribe CSV?
Se intercambian `density` y `alcohol`	Validación del manifiesto, antes de `joblib.load`	`Manifiesto incompatible: ... feature_names no coincide`	No
Cambia `preprocessing_version`	Validación del manifiesto, antes de `joblib.load`	`Manifiesto incompatible: ... preprocessing_version no coincide`	No
El modelo devuelve `unknown`	Inferencia, al validar la salida del estimador	`quality_band no permitida por el manifiesto: 'unknown'`	No
La petición incluye `ph = 99`	Validación de la fila con `WineQualityRequest`	`Error en la fila 0 (red-001): ... ph: Input should be less than or equal to 4.5`	No
Cambia `estimator_type` (el manifiesto declara otro tipo)	Carga del bundle, justo después de `joblib.load` y antes de inferir	`estimator_type no coincide: RandomForestClassifier !=`	No

Cambio	Frontera que debe rechazarlo	Mensaje útil esperado	¿Se escribe CSV?
		`LogisticRegression`	

El tipo de estimador es la única comprobación que necesita el objeto ya deserializado, porque hay que compararlo con el tipo real; aun así, se hace antes de cualquier predicción.

Invariantes

Condiciones que deben cumplirse; si alguna falla, se bloquea la carga o la inferencia y no se escribe ningún resultado:

Invariante	Comprobación	Error esperado	Etapa de rechazo
El bundle contiene `manifest.json` y `model.joblib`	Existencia de ambos ficheros	`El manifiesto no existe` / `El modelo no existe`	Carga, antes de leer nada
El manifiesto tiene exactamente las seis claves y es compatible con el código	`ArtifactManifest` con `extra="forbid"` y validadores de `schema_version`, `preprocessing_version`, `feature_names` y `output_labels`	`Manifiesto incompatible: ...`	Carga, antes de deserializar
El estimador cumple la interfaz mínima y es del tipo declarado	`isinstance(estimator, WineQualityEstimator)` y `type(estimator).__name__ == estimator_type`	`El estimador no cumple la interfaz mínima (predict/predict_proba)` / `estimator_type no coincide`	Carga, tras deserializar y antes de inferir
La salida pertenece a `output_labels` y la confianza está entre 0 y 1	Comprobación en `infer_wine_quality` y `WineQualityPrediction`	`quality_band no permitida por el manifiesto`	Inferencia, antes de escribir el CSV

5. Práctica 4 — Preparar la implementación de la segunda clase

Antes de pasar al taller, deja cerradas estas decisiones:

- ¿Qué campos son obligatorios y cuáles tienen valores por defecto?
- ¿Aceptarás campos desconocidos en el manifiesto?
- ¿Validarás el manifiesto antes de cargar el objeto serializado?

- ¿El CSV de salida se escribe fila a fila o solo después de validar todas las entradas?
- ¿Qué mensaje verá la persona que ejecuta la CLI cuando el contrato falle?
- ¿Qué ocurre si una segunda fila es inválida después de haber procesado una primera fila correcta?

La segunda clase convertirá estas decisiones en **ArtifactManifest**, **save_model_bundle**, **load_model_bundle**, inferencia y pruebas.

Decisiones

- **Campos obligatorios y valores por defecto.** Las seis claves del manifiesto son obligatorias y no tienen valor por defecto. Al crear el manifiesto, `preprocessing_version`, `feature_names` y `output_labels` se toman de las constantes del código, y `estimator_type` del tipo real del estimador; solo `model_version` se indica explícitamente.
- **Campos desconocidos.** No se aceptan (`extra="forbid"`): un campo que el código no conoce indica que el bundle se generó con otro esquema.
- **Orden de validación.** Sí: primero se valida `manifest.json` y solo después se ejecuta `joblib.load`. La interfaz y el tipo del estimador se comprueban tras deserializarlo, pero antes de cualquier predicción.
- **Escritura del CSV.** Solo al final: se validan todas las filas, se acumulan las predicciones en memoria y se escribe en un fichero temporal que se renombra al destino con `os.replace`.
- **Mensaje para quien ejecuta la CLI.** Un mensaje que indica qué ha fallado y dónde (fila, `sample_id` y campo, o el campo del manifiesto), y código de salida 2.
- **Segunda fila inválida.** Se aborta todo el procesamiento y no se publica ningún fichero, aunque la primera fila fuera correcta.